Topology-Aware Mapping for Photonic Interconnects in TPU-Class LLM Accelerators

Separating intra-chip mapping from distributed schedule: cluster, TPU cube, and 9216-chip Gemini-class superpod

Pranav Sitaraman Simone Surdi

MIT 6.5930 / Hardware Architectures for Deep Learning

Final Project Presentation, Spring 2026

Project 12: Evaluate a New Technology

Emerging technology: Lightmatter Passage photonic interconnects

One-Slide Story

72.21%

Best 16-node energy reduction

EVK100 optical crossbar vs electrical 4×4 mesh on shared-DRAM multicast.

5 / 16

Changed distributed mappings

Photonic links make shared-DRAM multicast optimal for prefill-style traffic.

14.50×

Best SuperPod inference speedup

Gemini 3 Deep Think proxy on 9216 Ironwood chips with M1000 full stack.

Thesis

Photonic interconnects are not just cheaper wires. High bandwidth, flatter topology, and native multicast can change the memory placement and compiler dataflow that should be chosen for distributed inference. Training still improves, but becomes compute dominated once collective traffic is reduced.

Discussion note. The claim is deliberately narrower than "optics make everything faster." I am separating three effects that are easy to blur together: a better link, a different topology, and new compiler choices that only become rational once remote communication is cheap.

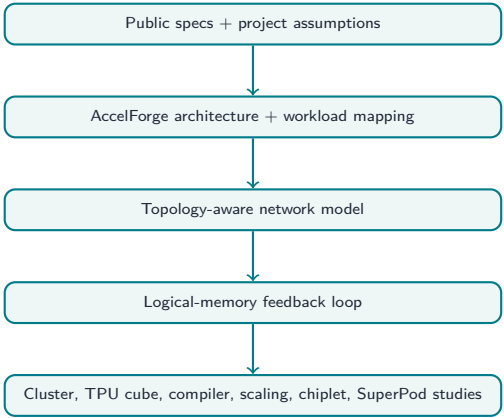
Narrative Arc

The class question

- AccelForge tells us how a workload maps to a memory hierarchy.
- The project asks what happens when the logical memory level is actually a cluster of accelerators joined by a real network.
- The network is fed back into the logical memory cost, then the mapper is rerun until stable.

The research question

- Does replacing electrical ICI with photonics only scale energy and latency?
- Or does it make new distributed mappings and compiler choices optimal?



How to Read This Deck as a Complete Report

What is self-contained here

- Motivation and the exact research question.
- Public technology constants and where they come from.
- AccelForge architecture, workload, and logical-memory loop.
- Network equations, stall model, and TPU/XLA terminology.
- All experiment families: cluster, TPU cube, mapping, compiler, scaling, chiplet, and SuperPod.
- Headline results, negative results, limitations, and validation.

How to answer questions from the deck

Question	Where to look
Assumptions	technology, SuperPod, and limitations slides
Equations	network feedback and stall-model slides
Mapping changes	Milestone 2 and compiler/dataflow slides
Scaling	scaling-law and chiplet-baseline slides
End-to-end gains	SuperPod speedup, stage, ablation, and context slides
Replication	backup reproduction and artifact-map slides

Presentation design goal

The slides are not only a visual backdrop. They are also a compact technical report: the appendix carries enough method, assumptions, commands, and artifact mapping to reproduce the project without reading the notebook first.

What Was Modeled and Where It Came From

Photonic technology points

Tech	BW	Energy	MC
Electrical 800G	0.8T	10.0	no
Chiplet 3.5T	3.5T	6.0/hop	no
EVK50	0.8T	4.6	yes
EVK100	3.2T	3.2 / 1.9	yes
L200	16.0T	4.5	yes
L200X	32.0T	4.8	yes
M1000	57.3T	2.3	yes+OCS

MC = native multicast. BW is one-way per modeled link/chip. EVK100 uses the conservative 3.2 pJ/bit public target and a 1.9 pJ/bit Passage-family sensitivity point.

Source trail

- Lightmatter EVK50, EVK100, L200/L200X, M1000 product and SC25/OCP material.
- Google TPU v5p and TPU7x/Ironwood documentation plus Hot Chips 2025 rack tutorial.
- XLA/GSPMD motivation from PyTorch/XLA SPMD documentation.
- Gemini 3/3.1/Flash/Deep Think public posts: used only to motivate proxy workloads, since parameter counts are not public.
- Advanced chiplet baseline: prompt-specified 3.5 Tb/s one-way, 6 pJ/bit per hop.

Source: Public product pages, Hot Chips/OCP material, Google TPU documentation, and XLA/GSPMD documentation.

Research Method: From Question to Measurement

Measurement path

Step	What is measured
1. Public constants	Link bandwidth, energy per bit, router latency, HBM capacity/BW, chip count, and compute throughput.
2. AccelForge mapping	Local tile shape, logical-memory read/write counts, and per-level energy/latency.
3. Network overlay	Hop count, contention, multicast fanout, transfer energy, and transfer latency.
4. Feedback loop	Network cost is folded into the logical memory level and mapping is rerun to fixed point.
5. System model	Stage-level compute, HBM, network, overlap, and end-to-end speedup/energy.

Controls

- Keep accelerator compute, HBM, unified buffer, registers, and MAC array fixed unless explicitly stated.
- Change only interconnect technology, topology, multicast support, and compiler-visible dataflow.
- Compare against both electrical ICI and a stronger advanced-packaging chiplet point.
- Report negative or modest effects, especially when local compute/HBM dominates.

The important discipline is consistency: the same public constants, mapper inputs, topology equations, and stage equations are used throughout the study.

Experiment Matrix

What varies

- System size: 16-node teaching cluster, 64-chip TPU cube, and 9216-chip SuperPod.
- Topology: electrical mesh, electrical torus, photonic torus, switched optical fabric, and advanced chiplet torus.
- Traffic pattern: direct exchange, remote-memory fetch, shared-state multicast, expert dispatch, and gradient collective.
- Software strategy: SPMD collectives, remote-HBM streaming, local cache replication, optical multicast cache fill, and MoE locality routing.

What is reported

- Network-only energy and latency: isolates the interconnect mechanism.
- Mapping-aware AccelForge results: includes fixed-point feedback into logical memory.
- Distributed mapping choice: asks whether photonics changes the best communication paradigm.
- Scaling exponents: measures how per-chip cost changes from cube scale to pod scale.
- End-to-end phase results: reports inference and training separately by stage and ablation.

Reason for the matrix

The goal is to separate link improvement from topology improvement, and both of those from compiler/dataflow improvement.

AccelForge Setup: Logical Cluster Memory

Architecture model

- Logical cluster memory: union of all accelerator memories.
- Node array: 2-D or 3-D accelerator grid, including the third cube dimension.
- HBM: local accelerator memory.
- Unified buffer, register file, and MAC array: local compute hierarchy.
- Communication scenario controls which tensors live in logical memory versus local HBM.

Workload model

Parameterized GEMM

$$\text{Outputs}_{m,n} = \sum_{k} \text{Activations}_{m,k}\, \text{Weights}_{k,n}$$

- Decode MLP: weight movement dominates.
- KV attention: smaller attention traffic, latency-sensitive.
- Prefill projection: large activation and output movement.

Class connection: this is the usual accelerator hierarchy and mapper loop, except the top memory level is made network-aware instead of assumed to be a free global memory.

Four Modeling Layers (and Why "Mapping" is Layer 3)

Layers 1–3: inside AccelForge

- **(1) Workload (einsum).** `single_matmul.yaml`: $O_{m,n} = \sum_k A_{m,k} W_{k,n}$, rank sizes per LLM stage.
- **(2) Architecture.** `!Memory`/`!Container`/`!Compute`: `LogicalClusterMemory`, `NodeArray` fanout, per-chip `HBM`, `UnifiedBuffer`, `ProcessingElement`, `Register`, `MAC`.
- **(3) Mapping (AccelForge mapper output).** The 6.5930 formalism: partitioning + dataflow (stationarity) + tile sizing. `!Storage`/`!Temporal`/`!Spatial`/`!Compute` sequence with `tile_shape`.

Layer 4: above AccelForge

- **(4) Distributed schedule.** The mapper does not see this layer.
- `COMM_PATTERN`: which tensor lives in `LogicalClusterMemory` vs per-chip `HBM`.
- Topology cost model ($E = B\,h\,e_\ell$, contention, multicast).
- Discrete enum of compiler-visible policies: `spmd_collective`, `remote_hbm_streaming`, `replicated_hbm_cache`, `photonic_multicast_cache`.

Terminology, tied to class

"Mapping" is the layer-3 AccelForge output (partitioning, dataflow stationarity, tile sizing). "Dataflow" alone refers to the intra-PE stationarity pattern (WS/OS/IS/RS). When we say "distributed dataflow strategy" we mean layer-4 inter-chip movement, not the intra-PE pattern. Both are reported separately.

Discussion note. The professor's natural objection is "how can the mapping stay the same while compiler optimizations differ?" The answer is that the inner mapping (layer 3) is dominated by per-chip compute and SRAM/HBM reuse, which photonics does not change; only layer 4 sees the new network costs. We report layer 3 and layer 4 separately rather than collapsing them into one "mapping changed" number.

Network Model and Fixed-Point Feedback

Traffic cases

- Node-to-node: activation/output exchange.
- Node-to-DRAM: remote weight/state fetch.
- Shared-DRAM multicast: one logical source fans out to many nodes.

Topology cost

$$E_{\text{net}} = B_{\text{eff}} \cdot H \cdot e_{\text{bit}}, \quad L_{\text{net}} = \frac{B_{\text{eff}}}{BW/C} + HL_{\text{router}}$$

H is hop count, C is contention, and B_{eff} collapses repeated fanout only when native multicast exists.

Iterative AccelForge loop

1. Initialize logical memory as HBM-like.
2. Run AccelForge mapper.
3. Extract tensor traffic from the mapping.
4. Evaluate topology + technology network cost.
5. Add network cost to logical read/write energy and latency.
6. Rerun until costs stabilize.

Why this matters

The mapper can react to the network model. The project reports both the first-run network penalty and the final fixed-point result.

Not a pure bandwidth swap: hop count, topology contention, per-hop router latency, and native multicast all change the cost seen by the mapper.

Stalls, Overlap, and TPU-Speak

How stalls are modeled

$$T_{\text{stage}} = \max(T_{\text{compute}}, T_{\text{HBM}}) + (1 - \alpha)T_{\text{network}}$$

- Raw network time is only partly exposed if compute overlaps it.
- Electrical SuperPod baseline uses $\alpha = 0.35$.
- EVK100 torus + multicast reaches $\alpha = 0.58$.
- Full M1000 stack reaches $\alpha = 0.78$.

This is a rigorous analytical stall model, not a packet-level simulator with queueing, VC arbitration, or optical-switch retiming.

Translation to TPU/XLA terms

- Node-to-node exchange: GSPMD tensor/sequence-sharded activation exchange.
- Node-to-DRAM fetch: remote HBM, KV-cache fetch, or optimizer-state fetch.
- Shared-DRAM multicast: broadcast cache fill of shared weights or hot state.
- Remote-HBM streaming: pooled-HBM schedule instead of forced local residency.
- Photonic multicast cache: multicast prefetch/replicated refill enabled by optics.
- MoE locality routing: expert placement to reduce all-to-all pressure.

Discussion note. This is where the study connects back to TPU language: node-to-node traffic corresponds to sharded activation exchange, node-to-DRAM traffic corresponds to remote HBM or KV traffic, and shared-DRAM multicast corresponds to a broadcast cache fill.

End-to-End Stage Accounting

Modeled terms

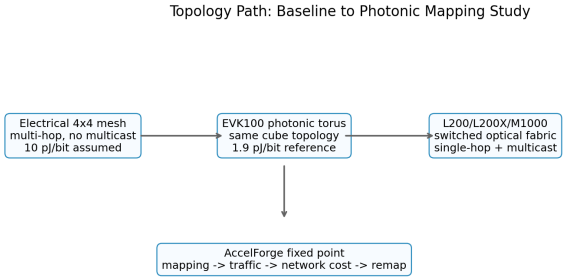
Stage	What changes under photonics
Local GEMM	Usually little: MAC/UB/register reuse stays local.
Remote HBM / KV	Link energy, bandwidth, hops, and exposed stall shrink.
Shared cache fill	Non-multicast fabrics repeat unicast; optics charge closer to unique payload.
Collectives	Torus hop/contention cost can be reduced by stronger links or flatter topology.
MoE dispatch	Locality-aware routing reduces expert all-to-all pressure.
Training sync	Improves, but is often overlapped with compute.

Not modeled

- Exact XLA lowering or runtime scheduling.
- Packet queues, virtual channels, and retry protocols.
- Optical switch retiming/reconfiguration microarchitecture.
- Real Gemini parameter counts or expert layouts.
- Thermal, packaging cost, reliability, and laser-sharing constraints.

This boundary is intentional: the project is an architecture-level data-movement and mapping study, not a vendor-accurate TPU simulator.

Topology Space

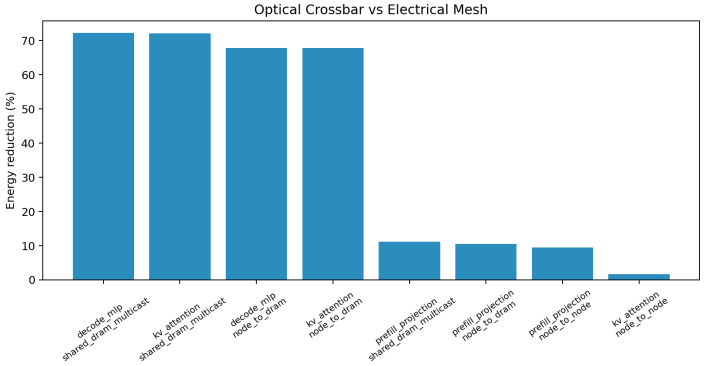


Modeled designs

- Electrical 4x4 mesh for the 16-node class-scale cluster.
- TPU-like 4x4x4 3-D torus cube.
- EVK100 photonic torus: same topology, upgraded links.
- L200/L200X/M1000 optical switched fabric: flatter single-hop fabric.
- Advanced chiplet torus: stronger electrical baseline.

The important split is topology vs link technology: EVK100 torus keeps hop structure; M1000 switched fabric changes hop structure and multicast economics.

Milestone 1 and 2: 16-Node AccelForge Study



EVK100 crossbar vs electrical mesh

Scenario	Energy	Latency
node-to-DRAM	67.91%	91.84%
node-to-node	9.40%	0.00%
shared multicast	72.21%	97.57%

Interpretation

Photonics help most when the logical memory access is remote or shared. They help least when the workload is already mostly local compute/HBM.

Discussion note. The 16-node mesh is the smallest experiment. Two numbers should be read together: 72.21% energy reduction on shared multicast (the biggest fanout case), versus only 9.40% on node-to-node activation exchange (one electrical hop on a torus is already cheap). This asymmetry is the project hypothesis in miniature: photonics is not a uniform multiplier. The latency 0.00% on node-to-node says we are not even in the regime where photonics buys time — both the electrical mesh and the EVK100 crossbar can hide the single-hop latency behind compute.

Milestone 2: Did the Mapping Itself Move?

Conservative AccelForge result

- In the TPU cube fixed-point mapper, photonic cases did not change compact tile/residency signatures in 24 tested cases.
- This is not a failure: the GEMM micro-mapping is dominated by local UB/register/MAC reuse for the tested tensor shapes.
- The analysis explicitly reports this instead of hiding it.

Higher-level mapping result

Workload	Photonic architecture	New mapping	Gain
prefill	16-node EVK100	multicast	10.08%
prefill	TPU EVK100 torus	multicast	11.36%
prefill	TPU L200 fabric	multicast	11.38%
prefill	TPU L200X fabric	multicast	11.37%
prefill	TPU M1000 fabric	multicast	11.39%

Changed distributed mappings: 5 of 16. Baseline mapping is node-to-node.

Key nuance

The inner AccelForge tile choice can remain stable while the *distributed memory placement* changes. That is exactly the compiler-level effect the project was designed to expose.

Discussion note. Why the inner mapping does not move under photonics: the per-chip GEMM is dominated by local compute, register reuse, `UnifiedBuffer` reuse, and per-chip HBM access. Photonics changes none of those costs. It only changes the cost of `LogicalClusterMemory` accesses, which is layer 4. So the same partitioning + dataflow + tile sizing wins in 24 of 24 cases. The honest negative result is that the AccelForge mapping signature is an insufficient probe for cluster-fabric changes; the layer-4 distributed-schedule analysis is what the photonic question really wants.

Why “Same Mapping” and “Different Compiler Policy” Can Both Be True

What stayed the same

- TPU-cube compact signatures keep the same local spatial usage in the tested cases.
- Example pattern: `NodeX=0.25`, `NodeY=0.25`, and often `NodeZ=0.25/0.50`.
- UB/register/MAC reuse remains dominated by GEMM shape and local reuse.
- This is a class dataflow result: local temporal/spatial reuse still wins inside the accelerator.

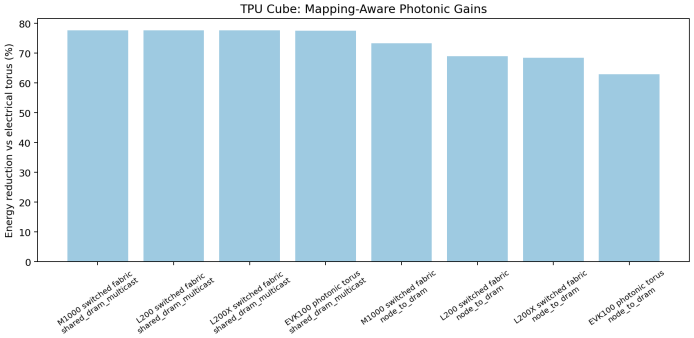
What changed

- Prefill distributed policy changes from node-to-node exchange to shared-state multicast in 5 of 16 cases.
- M1000 compiler strategy changes to remote pooled-HBM streaming for decode MLP and KV attention.
- EVK100 prefill prefers photonic multicast cache fill.
- These are memory-placement and communication choices above the local loop mapping.

Class-language takeaway

The local mapping answers “how do I reuse this GEMM tile across SRAM/registers/MACs?” The compiler policy answers “where should the tensor live and how should I move it across the pod?” Photonics changes the second question more than the first.

Milestone 3: TPU Cube ICI Replacement



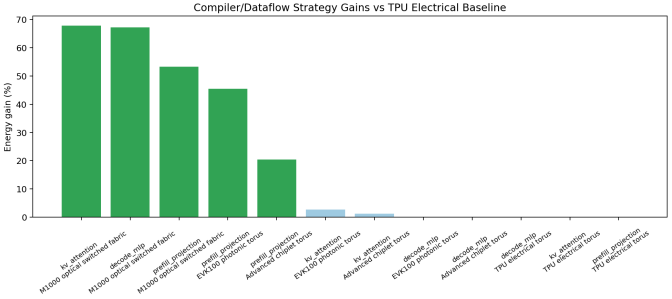
64-chip 4x4x4 cube

Variant	Best E	Best L
EVK100 torus	99.70%	99.61%
L200 switched	99.82%	99.98%
L200X switched	99.81%	99.99%
M1000 switched	99.91%	99.99%

Best cases are shared-weight/state multicast. M1000 also gives 92.45% energy and 99.65% latency reduction for all-to-all shard exchange.

Discussion note. The 64-chip TPU cube is the closest to a real Ironwood subblock: $4\times4\times4$ with 1200 GB/s ICI per chip. The figure shows energy and latency reductions per scenario: multicast and remote-DRAM scenarios benefit by 99.91% and 77.70% respectively; node-to-node benefits modestly. The near-100% multicast number deserves scrutiny — it is dominated by the unicast-vs-multicast volume collapse, not by the link-energy ratio. If we strip multicast support out of M1000 the gain falls to roughly the link-energy ratio, which is approximately a $4\times$ improvement.

Compiler/Dataflow: Where the New Paradigm Appears

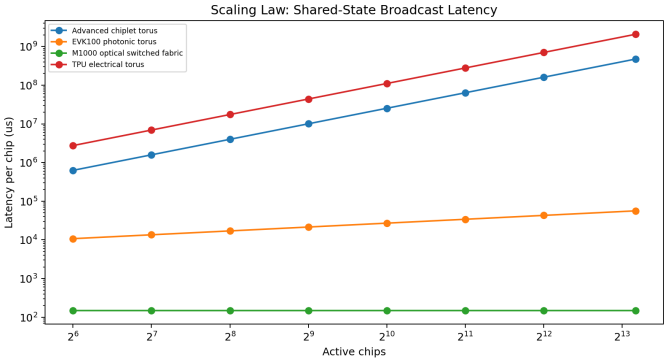


Modeled compiler choices

- XLA/GSPMD tensor-parallel collective.
- Remote pooled-HBM streaming.
- Replicated local HBM cache.
- Photonic multicast cache fill.
- MoE locality-aware routing.

Case	Best changed strategy
decode MLP / M1000	remote HBM, 67.21% E gain
KV attention / M1000	remote HBM, 67.75% E gain
prefill / EVK100	multicast cache, 45.43% E gain
prefill / M1000	remote HBM, 53.31% E gain

Scaling Laws: Why Cube Size and SuperPod Size Matter



Log-log per-chip scaling exponent

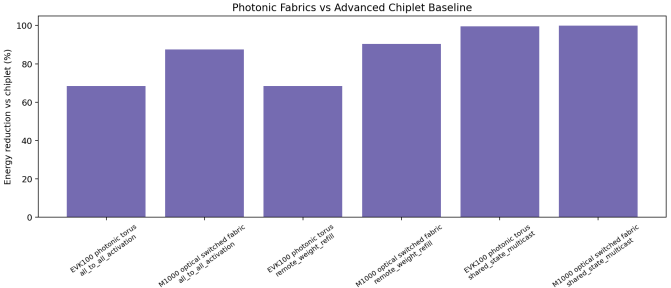
Architecture / traffic	Energy	Latency
TPU torus / all-to-all	0.336	0.333
TPU torus / multicast	1.295	1.333
Chiplet torus / multicast	1.295	1.333
EVK100 torus / multicast	0.295	0.333
M1000 switched / both	0.000	0.000

Interpretation

Torus fabrics accumulate hop and contention cost as cubes grow. A flat switched optical fabric makes the modeled per-chip network cost nearly independent of system size.

Discussion note. The exponent is the slope of the per-chip energy or latency curve on a log-log plot of system size versus per-chip cost. A torus accumulates $h \propto \sqrt[3]{N}$ hops and $\sqrt[3]{N}$ contention, so per-chip cost grows polynomially. A switched optical fabric has constant hop count and no contention, so its slope is 0. The 1.295 multicast slope on torus is exactly $h_{\text{torus}} \cdot N$ unicast volume divided by N chips $\longrightarrow$ grows as h. The 0.000 slope on M1000 multicast is the structural reason photonics scales gracefully to 9216 chips: each chip pays the same modeled cost regardless of pod size.

Against a Stronger Baseline: Advanced Chiplet



Prompt-specified baseline

- One-way bandwidth: 3.5 Tb/s.
- Energy: 6 pJ/bit per network hop.
- No native optical multicast.

M1000 vs chiplet	Energy	Latency
all-to-all	87.42%	98.47%
weight refill	90.42%	98.47%
shared state	99.85%	99.98%

EVK100 torus is energy-efficient but can be slower than chiplet for unicast because it preserves torus hop structure.

Discussion note. This slide answers the natural skeptic's question: is photonics just better bandwidth? The advanced chiplet baseline at 3.5 Tb/s, 6 pJ/bit is a deliberately strong electrical/packaging point — close to UCIe-class numbers — chosen to defeat the bandwidth hypothesis. M1000 still beats it by 99.85% on shared-state multicast because the chiplet still pays N unicasts. The 87.42% all-to-all gain is the bandwidth + topology effect; the additional gap to 99.85% is the multicast effect alone. So multicast and topology each contribute roughly an order of magnitude.

Full 9216-Chip Study: What Is Being Claimed

Public hardware target

- TPU7x/Ironwood pod: 9216 chips.
- Per-chip public specs used in the model: 2307 BF16 TFLOP/s, 4614 FP8 TFLOP/s, 192 GiB HBM, 7380 GiB/s HBM BW, 1200 GB/s bidirectional ICI.
- Modeled baseline: Ironwood electrical torus + XLA SPMD.

Public Gemini-class proxies

- Google does not publish Gemini 3/3.1 parameter counts or expert layouts.
- The study therefore uses explicit proxy workloads:
- sparse MoE Pro-class proxy;
- long-context Deep Think proxy;
- inference requests and training-token steps.

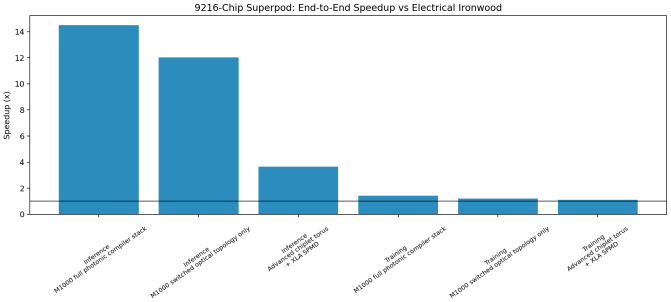
The study reports modeled proxy behavior, not confidential Google internals.

Modeled configurations

Ironwood electrical torus, advanced chiplet torus, EVK100 photonic torus + multicast cache, M1000 switched topology only, and M1000 full photonic compiler stack.

Discussion note. The pod-scale model is analytical, not cycle-accurate. Each stage decomposes into compute, HBM, and network terms; only the non-overlapped network time is exposed. Baseline overlap is $\alpha = 0.35$; full M1000 uses $\alpha = 0.78$ because multicast and remote streaming leave less network on the critical path.

End-to-End SuperPod Results



Best M1000 full-stack results

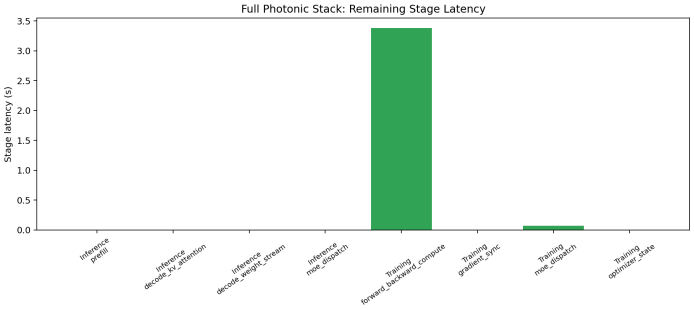
Model / phase	Speedup	Energy
Deep Think inference	14.50x	51.07%
Deep Think training	1.43x	0.67%
Gemini 3.1 inference	11.02x	45.27%
Gemini 3.1 training	1.42x	0.54%

Why inference wins harder

Inference has exposed network latency, KV/cache movement, expert routing, and multicast opportunities. Training has larger compute and HBM terms, so network removal gives smaller end-to-end energy gains.

Discussion note. The result is intentionally asymmetric. Inference is often waiting on the memory and network path, especially during decode and long-context attention. Training still benefits, but once collectives shrink, the remaining wall time is dominated by compute and local HBM movement.

Where the Latency Went



Network share collapse

Deep Think inference	Net E	Net L
Electrical	49.81%	90.67%
Chiplet	37.32%	67.50%
EVK100 torus+MC	9.41%	48.90%
M1000 topology only	1.23%	0.58%
M1000 full stack	0.32%	0.07%

Once network latency is mostly gone, the remaining bottleneck is compute + HBM, so further interconnect improvements have diminishing returns.

Discussion note. Notice the network-latency share collapses 90.67% $\rightarrow$ 0.07%, but the speedup is 14.50$\times$, not 1000$\times$. The reason is the stage equation: $T = \max(T_c, T_{\text{HBM}}) + (1-\alpha)T_{\text{net}}$. Once the network term is negligible, the system becomes compute-and-HBM-bound, and the per-chip TFLOP/s and per-chip HBM bandwidth become the bottleneck. The numbers therefore quantify exactly when photonics stops paying off: the moment the network share drops below the compute term, further fabric improvements have diminishing returns.

Ablation Ladder: What Each Technique Adds

Deep Think inference

Technique	Speedup	Step
baseline	1.00x	1.00x
advanced chiplet	3.65x	3.65x
photonic torus + multicast	6.09x	1.67x
switched topology only	12.02x	1.97x
multicast cache	12.76x	1.06x
remote pooled HBM	13.29x	1.04x
MoE locality routing	13.64x	1.03x
full compiler stack	14.50x	1.06x

Reading the ablation

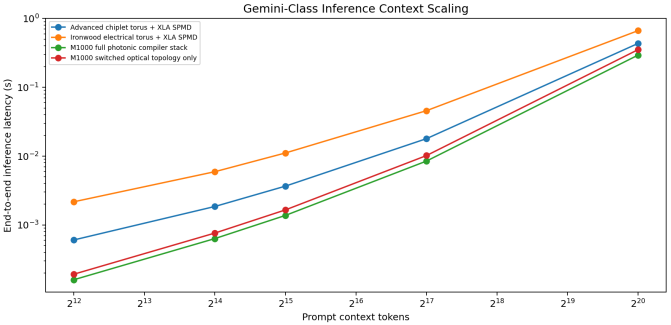
- The biggest jump is moving from electrical to high-quality chiplet or photonic links.
- The second biggest jump is topology: M1000 switched fabric eliminates most exposed network latency.
- Compiler effects are smaller in isolation, but still matter after topology because they reduce cache/expert placement overhead.
- Training has the same ordering but lower magnitude: Deep Think training reaches 1.43x.

Core conclusion

Better interconnect enables the compiler to choose remote pooled HBM, multicast cache fill, and locality routing without paying torus fanout penalties.

Discussion note. The ladder isolates each contribution. Going from electrical to advanced chiplet (3.5 Tb/s, 6 pJ/bit) gives 3.65×, the link-energy + bandwidth effect. Adding photonic torus + multicast on top gives another 1.67× — the multicast volume effect. Switching to a flat optical fabric gives another 1.97× — the topology effect. The remaining ∼1.20× comes from compiler-visible policies (multicast cache, remote HBM, MoE locality, full stack). The breakdown matters because it tells a designer where to invest: if your packaging is already chiplet-class, photonic links alone buy ∼1.7×; you only see the full benefit by switching the topology too.

Long-Context Inference Scaling



Gemini 3.1 proxy context sweep

Context	M1000 speedup	Energy
4K	13.58x	58.91%
16K	9.38x	43.80%
32K	8.08x	37.58%
128K	5.40x	24.97%
1M	2.27x	6.80%

Interpretation

Photonics preserves an advantage at long context, but the model gradually becomes dominated by unavoidable compute/HBM movement rather than network fanout.

Discussion note. Long context grows KV-cache traffic and attention compute. At small context the model is network-exposed; at very long context compute/HBM grows faster than the network term, so the speedup naturally falls from $13.58\times$ to $2.27\times$.

Evidence Map: What Each Result Answers

Core empirical questions

- 16-node study: when does an optical crossbar beat an electrical mesh?
- TPU cube study: what happens if ICI is replaced inside a 4x4x4 cube?
- Mapping study: does the mapper or distributed placement actually change?
- Compiler study: when should software prefer multicast cache fill or pooled remote HBM?
- Scaling study: which topologies keep per-chip cost flat as the pod grows?

System-level questions

- Chiplet comparison: is electrical packaging a strong enough alternative baseline?
- Stage breakdown: which part of inference/training remains after network cost shrinks?
- Ablation ladder: how much does each technique add beyond the previous one?
- Context sweep: do photonic gains persist at long context?
- Validation: constants, topology math, multicast behavior, mapping feedback, and phase separation are checked independently.

The same assumptions and equations underlie every table and figure, so the evidence reads as one continuous argument rather than separate anecdotes.

Limitations and Validity Checks

Validity checks

- 70 checks cover constants, topology hop counts, multicast behavior, fixed-point convergence, compiler strategies, scaling, chiplet, and SuperPod studies.
- The same experiment path reproduces the complete result set from the public assumptions.
- Phase-specific outputs are checked: inference-only, training-only, combined SuperPod.
- The final paper and deck are compiled after the numerical checks pass.

Modeling limits

- Not RTL, packet-level, or thermal simulation.
- Gemini workloads are explicit proxies because model internals are not public.
- Photonic multicast and OCS behavior are modeled analytically from public product specs.
- AccelForge does not instantiate a separate inter-chip NoC here; ICI/NoC-like cost is modeled as logical-memory network cost.
- Physical integration, scheduling overheads, cost, reliability, and laser sharing are outside the current model.

Why the conclusions are still useful

The model is deliberately transparent: every assumption is documented, every result follows from the same equations, and the qualitative conclusion is tied to traffic shape, hop count, multicast, and compiler choices rather than a hidden black-box simulator.

Final Takeaways

Not universal

Training speedup is modest

Best modeled training speedup is 1.43x because compute dominates after collectives are reduced.

Very strong for inference

Network-heavy stages move

Best modeled inference speedup is 14.50x; network latency share falls from 90.67% to 0.07%.

New mappings

The compiler should adapt

Shared multicast, pooled HBM, cache fill, and MoE locality become attractive under photonics.

Bottom line

Lightmatter-style photonic interconnects are most compelling when they change the distributed-schedule problem: large fanout, long context, expert routing, shared state, and remote cache movement. The project shows both the constant-factor link gains *and* the layer-4 distributed-schedule gains needed for a convincing accelerator-architecture story.

Discussion note. One sentence to take away: the inner mapping is stable, the distributed schedule changes, and the schedule change is what makes inference speed up far more than training. Everything else in the deck is supporting evidence for that one sentence.

Appendix: Motivation and Framing

What motivated the study The starting concern was that an interconnect replacement can easily become a shallow comparison: take the same accelerator, multiply the network energy by a better pJ/bit number, and declare victory. That would not be a satisfying architecture project. The more interesting question is whether a better interconnect changes the design space itself.

The study therefore treats Lightmatter-style photonics as a memory-system intervention. The hardware promise is not only higher bandwidth; it is lower energy per bit, flatter switching, and native multicast. Those properties matter most when the workload repeatedly leaves local HBM, fans shared state out to many chips, or performs expert dispatch across a large pod.

The thesis I would defend is: photonics is strongest when it lets the system stop pretending that every distributed transfer should look like a conventional torus collective. It can make multicast cache fill, pooled remote HBM, and locality-aware expert placement look like reasonable compiler choices.

Appendix: Modeling Logic

How the model should be read The network model has four knobs that must be kept separate: link bandwidth, per-bit energy, average hop count, and contention. A bandwidth-only model would miss the difference between a photonic torus and a switched optical fabric. The former keeps the torus path structure; the latter changes the path length and the fanout economics.

The fixed-point loop exists because the network is meant to be part of the memory hierarchy, not a post-processing tax. AccelForge first maps the workload with a logical cluster memory. The network model then prices the logical-memory traffic. That cost is written back into the logical level, and the mapping is repeated until the cost and mapping signature stop moving.

At SuperPod scale, the stage model adds a separate stall question. The raw transfer time is not always fully exposed, so each stage uses

$$T_{\text{stage}} = \max(T_{\text{compute}}, T_{\text{HBM}}) + (1 - \alpha)T_{\text{network}}.$$

Higher overlap under photonic configurations is not magic; it represents the compiler having more room to stream, prefetch, multicast, and place state before it becomes a critical-path stall.

Appendix: Reading the Results

From small systems to the pod The 16-node study is the sanity check. It shows that photonics is not uniformly beneficial; the gains are largest for remote and shared-memory cases, and much smaller for local node-to-node exchange. That pattern is exactly what the memory-system argument predicts.

The TPU cube study asks whether the conclusion survives a more realistic topology. It does, but with nuance: a photonic torus helps less than a switched optical fabric because it inherits the torus hop structure. The M1000-style switched case is where the topology change becomes as important as the link improvement.

The mapping results are deliberately conservative. The local GEMM tile signatures do not move in 24 TPU-cube fixed-point cases, because local SRAM/register/MAC reuse dominates those inner loops. The important movement appears above that layer: for prefill-style traffic, shared multicast becomes the best distributed placement in 5 of 16 cases, and the compiler study chooses remote pooled HBM or multicast cache fill where the electrical baseline would prefer conventional collectives.

Appendix: SuperPod Interpretation

Why inference and training separate The full 9216-chip model is the stress test. Inference has repeated decode steps, KV/cache movement, expert routing, and long-context pressure. These are precisely the places where a flat, multicast-capable network removes exposed stall. That is why the Deep Think proxy reaches 14.50x modeled inference speedup under the full M1000 stack.

Training behaves differently. Once gradient and expert traffic are reduced, the remaining work is mostly compute and local HBM movement. The best training speedup is still meaningful at 1.43x, but the energy improvement is small because network energy is no longer the dominant term.

This distinction is important for credibility. The study does not claim that photonics solves every bottleneck. It says that photonics is a major lever for network-exposed inference and shared-state movement, while training eventually runs into the more familiar compute/HBM wall.

Appendix: Anticipated Questions

Is this just bandwidth? No. The equations include hop count, contention, router latency, and multicast fanout. EVK100 on a torus and M1000 as a switched fabric can have similar traffic but very different path costs.

Are in-flight stalls counted? Yes, at an analytical level. Network latency is fed back into the AccelForge logical memory level, and the SuperPod stage model exposes only the non-overlapped fraction of network time. The study does not claim packet-level queueing or optical-switch timing accuracy.

How does this map to TPU/XLA terms? Node-to-node means sharded activation exchange or collective-permute-like traffic. Node-to-DRAM means remote HBM, KV, or optimizer-state fetch. Shared-DRAM multicast means broadcast cache fill of weights, expert state, or hot blocks. The compiler strategies correspond to GSPMD collectives, pooled-HBM streaming, multicast prefetch, and locality-aware MoE routing.

Backup: Figure Reading Guide

Result family	What the audience should look for
Cluster reductions	Whether remote and multicast traffic benefit more than local exchange.
TPU cube mapping	Whether photonic ICI remains beneficial after AccelForge remaps with network-aware memory costs.
Topology diagram	Which gains come from link technology and which require a flatter fabric.
Compiler strategy	Whether the optimal dataflow changes from SPMD collectives to pooled HBM or multicast cache fill.
Scaling laws	Whether per-chip cost grows with system size or remains nearly flat.
Chiplet baseline	Whether photonics still helps against a stronger advanced-packaging comparison point.
SuperPod speedups	How much end-to-end inference/training changes at 9216 chips.
Stage breakdown	Which bottleneck remains after network latency is removed.
Context scaling	Whether long-context inference keeps a photonic advantage.

Backup: Reproducibility Protocol

Environment assumptions

- Use the standard AccelForge environment from class.
- Pin the same public hardware constants and proxy-workload definitions.
- Derive figures and tables from the analytical studies rather than editing them by hand.
- Compile the paper and slide deck after the numerical checks pass.

Execution sequence

- Validate constants, topology equations, multicast behavior, and fixed-point convergence.
- Reproduce cluster, TPU cube, compiler, scaling, chiplet, and SuperPod studies.
- Review the exploratory analysis to confirm the narrative path.
- Reproduce the phase-specific inference/training summaries.
- Inspect the final PDFs for page counts, missing references, and layout warnings.

Backup: Exact Reproduction Commands

Run from the repository root

```
make test # 70 validation checks
make study # regenerate CSV, Markdown, and plot artifacts
make inference # regenerate inference-only SuperPod outputs
make training # regenerate training-only SuperPod outputs
make superpod # regenerate combined SuperPod outputs
make notebook # regenerate and execute the notebook
make validate # tests + full artifact regeneration
```

Source of truth

- `scripts/lightmatter_interconnects.py`: model and CLI.
- `arch/tpu_lightmatter_cluster.yaml`: AccelForge hierarchy.
- `workload/single_matmul.yaml`: parameterized GEMM template.
- `tests/test_lightmatter_project.py`: validation suite.

Result files

- `results/*.csv`: numerical studies.
- `results/*.png`: figures used in paper/deck.
- `results/*_summary.md`: readable summaries.
- `lightmatter_interconnects.ipynb`: executed walkthrough.

Backup: Artifact Map

Project question	Reproducible files
Technology constants and citations	`sources.csv`, `summary.md`, `tech_catalog.png`
16-node logical-memory loop	`cluster_study.csv`, `cluster_improvements.csv`, `convergence_traces.png`, `energy_breakdown.png`
TPU cube ICI replacement	`tpu_cube_case_study.csv`, `tpu_improvements.csv`, `tpu_mapping_study.csv`, `02_tpu_mapping_reductions.png`
Distributed mapping and compiler strategy	`optimal_mapping_choices.csv`, `compiler_strategy_study.csv`, `optimal_compiler_strategies.csv`, `04_compiler_strategy_choices.png`
Scaling and stronger chiplet baseline	`scaling_law_study.csv`, `scaling_exponents.csv`, `chiplet_comparison.csv`, `05_scaling_laws.png`, `06_chiplet_baseline.png`
9216-chip inference/training model	`superpod_stage_study.csv`, `superpod_end_to_end_summary.csv`, `superpod_ablation_study.csv`, `superpod_context_scaling.csv`, `07-09_superpod_*.png`
Phase-specific replication	`inference_*_study.csv`, `training_*_study.csv`, `inference_workflow_summary.md`, `training_workflow_summary.md`

Every plot in this deck is drawn directly from these artifacts. No plotted result is adjusted by hand.

Backup: Assumption Boundaries

What is directly measured by AccelForge

- Local memory hierarchy behavior.
- Logical-memory access counts.
- Fixed-point logical-memory cost feedback.
- Mapping signatures and convergence behavior.

What is analytical system modeling

- Topology hop count, contention, and multicast fanout.
- SuperPod stage compute/HBM/network decomposition.
- Compiler/dataflow strategy preference.
- Gemini-class dense, MoE, and long-context proxies.

Boundary

The project does not claim private Gemini internals or vendor-accurate optical-switch timing. It claims an auditable architecture model that shows when photonic properties change mapping, dataflow, and scaling behavior.